

ALGORITMOS y ESTRUCTURA DE DATOS II

Recursividad. Parte II



Recursividad vs Iteración



La iteración y la recursividad cumplen con el mismo objetivo: ejecutar un bloque de sentencias ***n*** veces y que con una condición de fin adecuada lo termine.



```
Function
  Factorial(n:integer):integer;
Var
  fac, i:integer;
Begin
  fac:=1;
  For i:=1 to n do
    fac:=fac*i;
  Factorial:=fac;
End;
```

Recursión en comparación con iteración

Ambas se basan en una estructura de control

- La iteración se vale de una estructura de repetición.
- La recursión emplea una estructura de selección.

Ambas se basan en la repetición

- La iteración utiliza explícitamente una estructura de repetición.
- La recursión logra la repetición a través de llamadas repetidas a una función.

Recursión en comparación con iteración

Ambas tienen una prueba de terminación

La iteración termina cuando la condición de terminación del ciclo falla.

La recursión termina cuando se reconoce un caso base.

Ambas pueden suceder infinitamente

Ocurre un ciclo infinito en la iteración si la prueba de continuación del ciclo nunca se vuelve falsa.

La recursión infinita sucede si no se cumple con el caso base.

.

Iteración	Recursividad
• Instrucción de repetición explícita • Termina cuando falla la condición • Repetición controlada por un contador • Infinita cuando la condición nunca se vuelve falsa	• Llamadas repetidas a los subprograma • Termina cuando se reconoce un caso base • Infinita cuando no se reconoce el caso base

Conclusión

Cualquier problema que puede resolverse con recursión también puede resolverse por iteración. Normalmente se prefiere un enfoque recursivo cuando refleja de manera más natural el problema y su resultado es un programa más fácil de entender y depurar.

Otra razón para seleccionar la solución recursiva es que tal vez no se encuentre una solución iterativa aparente.

Debemos evitar utilizar recursión en situaciones en las que es importante la eficiencia. Las llamadas recursivas tardan y consumen memoria.

Recursión:

Ventajas

- Menos líneas de código
- Refleja el problema con más naturalidad
- Produce un programa más fácil de entender y depurar

Desventajas

- Tiempo de procesador
- Espacio en memoria, consume memoria adicional

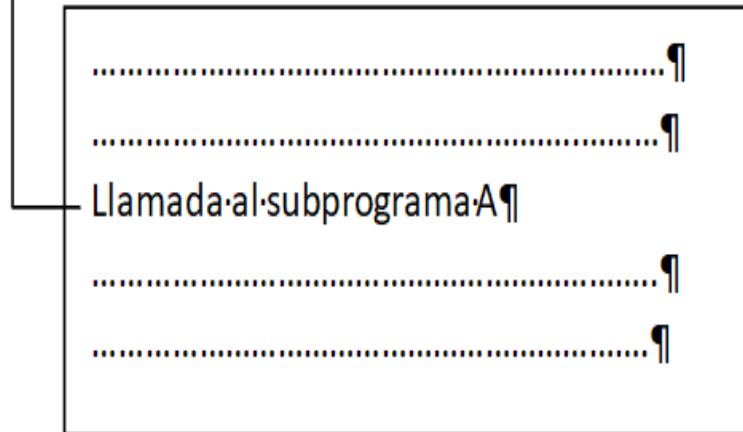


Recursión Directa e Indirecta

Directa:

El subprograma se llama directamente a si mismo.

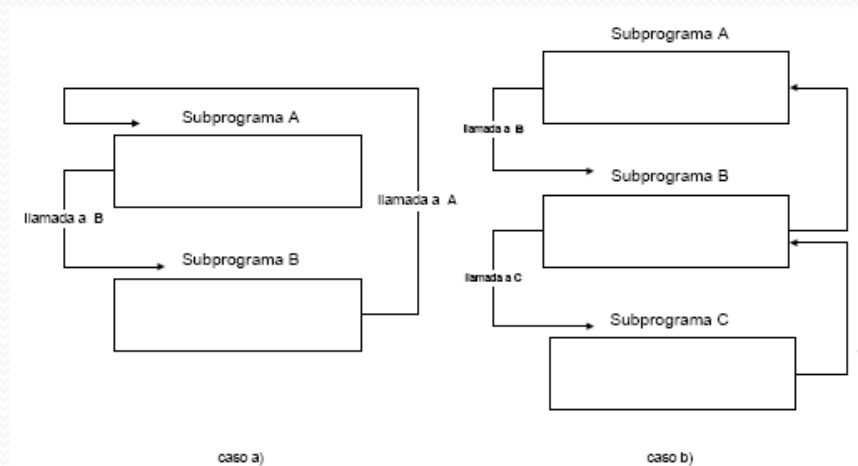
Subprograma A¶



Indirecta:

Un subprograma llama a otro subprograma y este a su vez al primero

El subprograma A llama al B y este a su vez llama al primero A, es decir que luego de tantas llamadas el control regresa al subprograma A.

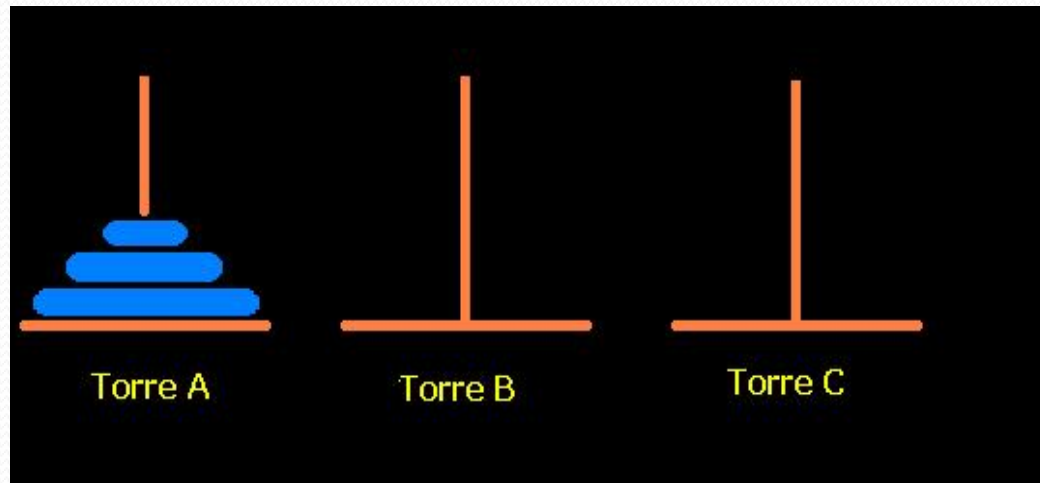




Caso típico de resolución de un problema con un método recursivo

El famoso juego de niños conocido como Torres de Hanoi

Consiste en tres varillas verticales y un número indeterminado de discos que determinarán la complejidad de la solución. No hay dos discos iguales, están colocados de mayor a menor en la primera varilla ascendentemente, y no se puede colocar ningún disco mayor sobre uno menor a él en ningún momento..



Aplicaciones

La recursividad es un método poderoso su poder radica en que algunos conceptos complejos pueden expresarse en una forma simple.

Las fórmulas recursivas pueden aplicarse a situaciones tales como prueba de teoremas, solución de problemas combinatorios, algunos acertijos, etc.